



Školska godina 2025/26

PREDMET:

CS320 - Operativni sistemi

Projektni zadatak

UPRAVLJANJE PROCESIMA U KLASITER OKRUŽENJU

Ime i prezime: **Luka Golubović**

Smer: **Softversko Inženjerstvo**

Broj indeksa: **6356**

Datum izrade: **13.08.2026.**

Asistent: **Goran Stamenović**

Sadržaj

Sažetak	3
2. Uvod i teorijska postavka izabrane teme	3
2.1 Procesi i raspoređivanje	3
2.2 Klaster kao skup ograničenih čvorova	4
2.3 Cilj rada	4
2.4 Metodologija	5
3. Primer/studija slučaja: implementacija simulatora i eksperimentalna analiza	5
3.1 Zahtevi i granica rešenja	5
3.2 Arhitektura aplikacije	6
3.3 Domenski model procesa	8
3.4 Politike raspoređivanja	9
3.5 Jedan takt simulacije	10
3.6 Kvar čvora i ponovno raspoređivanje	11
3.7 REST API i komandni alati	13
3.8 Konfiguracija i isporuka	15
3.9 Metrike	16
3.10 Eksperimentalni scenariji	16
3.11 Rezultati	17
3.12 Tumačenje balansiranog i heterogenog opterećenja	20
3.13 Tumačenje preopterećenja i prioriteta	20
3.14 Tumačenje kvara čvora	20
3.15 Ponovljivost i funkcionalna provera	20
3.16 Testiranje	21
3.17 Korektnost i složenost	22
3.18 Bezbednosne i operativne granice	23
3.19 Ograničenja i dalji rad	23
4. Zaključak	23
5. Spisak literature	24

Sažetak

Upravljanje procesima u klasteru obuhvata prijavu posla, izbor računarskog čvora, praćenje izvršavanja i obradu kvara. Razvijena Go aplikacija simulira taj postupak nad čvorovima sa ograničenim CPU i memorijskim kapacitetom. Proces definiše zahteve za resursima, trajanje, prioritet i politiku ponovnog pokretanja, dok kontroler proverava da nijedna odluka ne prekorači raspoloživi kapacitet.

Implementacija obuhvata deterministički simulator, REST servis i komandni klijent. Jezgro je podeljeno na domenski model, kontroler klastera, raspoređivače, izvršni engine, metrike i pomoćne skladišne operacije. Podržani su Round Robin, Least Loaded i Priority-Aware algoritmi. Kontroler obrađuje ručne i zakazane kvarove, heartbeat rokove, ograničene ponovne pokušaje, ponovno raspoređivanje, simulirani I/O, pauziranje i prekid procesa.

Studija slučaja koristi pet verzionisanih scenario fajlova i sedam referentnih pokretanja. Sedam pokretanja obuhvata balansirano, heterogeno, preopterećeno i prioritetno opterećenje, kao i kvar čvora. U svim referentnim pokretanjima pokrenuto je i završeno 100% prijavljenih procesa. Balansirano opterećenje završava se za 9 taktova sa oba poređena raspoređivača, ali Least Loaded smanjuje srednju standardnu devijaciju opterećenja sa 0,1393 na 0,0987. Preopterećen scenario dostiže prosečnu CPU iskorišćenost od 89,10%, uz 82 odlaganja raspoređivanja i prosečno čekanje od 11,25 taktova. Scenario kvara završava pet procesa za 16 taktova, sa dva ponovna pokretanja i dva ponovna raspoređivanja.

2. Uvod i teorijska postavka izabrane teme

Proces je program u izvršavanju sa sopstvenim stanjem, resursima i tokom obrade. Operativni sistem održava podatke o procesu, određuje kada može da se izvršava i oslobađa resurse kada se proces završi. Kada više procesa traži ograničen procesor ili memoriju, odluke raspoređivača utiču na vreme čekanja, propusnost i iskorišćenost sistema (Silberschatz, Galvin i Gagne, 2014; Tanenbaum i Bos, 2015).

Klaster je skup računara koji zajednički izvršava prijavljene poslove. Raspoređivanje u takvom sistemu nije ograničeno na izbor sledećeg procesa za jedan procesor, već obuhvata izbor dostupnog čvora sa dovoljnim resursima i praćenje njegovog stanja. Kvar čvora utiče na sve procese koji su mu dodeljeni, pa kontrolni sloj mora da utvrdi koji procesi smeju ponovo da se pokrenu i gde postoji slobodan kapacitet.

Sistemi za orkestraciju kontejnera rešavaju srodnu vrstu raspoređivanja. Kubernetes odvaja proveru izvodljivosti od izbora čvora: kandidati se filtriraju, ocenjuju i zatim vezuju za opterećenje (The Kubernetes Authors). Razvijena aplikacija nije zamena za Kubernetes. Njena namena je da isti problem predstavi modelom u kome se svaka odluka može ponoviti i objasniti.

Virtuelni takt je diskretna jedinica vremena simulacije. CPU i memorija predstavljani su celobrojnim jedinicama, a proces ne izvršava korisnički program. Ova granica uklanja mrežnu i hardversku neizvesnost kako bi analiza bila usmerena na predstavljanje stanja, izbor čvora, sprečavanje prekomerne alokacije i obradu kvara. Teorijski pojmovi zato su neposredno povezani sa domenskim modelom, algoritmima, interfejsima i rezultatima ponovljivih scenarija.

2.1 Procesi i raspoređivanje

Klasični model procesa obuhvata identitet, stanje izvršavanja, programski kontekst i resurse. U operativnom sistemu proces prelazi između spremnog, aktivnog, blokiranog i završnog stanja. Raspoređivač bira kada će proces dobiti procesor, dok mehanizmi sinhronizacije i I/O menjaju njegovu dostupnost (Tanenbaum i Bos, 2015).

Kriterijumi raspoređivanja zavise od namene sistema. Propusnost meri broj završenih poslova u vremenu. Vreme čekanja meri koliko dugo proces provodi spreman bez izvršavanja. Vreme obrta obuhvata period od prijave do završetka. Pravednost sprečava da pojedini proces čeka bez granice. Nijedan algoritam ne optimizuje sve kriterijume u svakoj raspodeli opterećenja (Silberschatz, Galvin i Gagne, 2014).

U klaster modelu proces je jedinica posla. Raspoređivač bira čvor, a ne jezgro procesora. Izvršavanje je nepreemptivno: kada proces dobije čvor, ostaje u stanju RUNNING dok ne završi, ne pređe u I/O

čekanje, ne bude pauziran ili prekinut, odnosno dok ga ne pogodi kvar. Vremenski kvant nije deo ulaznog ugovora. Round Robin kruži kroz čvorove prilikom smeštanja procesa i ne predstavlja CPU Round Robin sa podelom vremena.

2.2 Klaster kao skup ograničenih čvorova

Čvor ima CPU kapacitet, memorijski kapacitet, trenutno zauzeće i status. Novi proces može da se smesti samo na ONLINE čvor. Provera CanFit zahteva da slobodan CPU i memorija budu najmanje jednaki zahtevima procesa. Odluka se odbija čim jedan resurs nije dovoljan.

Status čvora može biti ONLINE, DRAINING, OFFLINE, FAILED ili RECOVERING. DRAINING čvor više ne prima nove procese jer raspoređivač prihvata samo ONLINE čvorove. Prelazak na OFFLINE dozvoljen je kada nema aktivnih procesa. Kvar odmah postavlja FAILED, uklanja alokacije i obrađuje pogođene procese.

Tabela 1 poredi klasično CPU raspoređivanje i model primenjen u simulatoru.

Tabela 1: Poređenje CPU i klaster raspoređivanja u realizovanom modelu.

Osobina	CPU raspoređivanje	Klaster raspoređivanje u aplikaciji
Kandidat	Proces ili nit	Proces kao jedinica posla
Odredište	CPU jezgro ili red izvršavanja	Čvor sa CPU i memorijskim kapacitetom
Ograničenja	prioritet, mogućnost preempcije i stanje procesa	status čvora, CPU, memorija i politika
Kvar resursa	prekid ili kvar lokalnog sistema	kvar čvora i obrada svih njegovih procesa
Metrike	čekanje, odziv, obrt, propusnost	čekanje, obrt, propusnost, iskorišćenost i balans

Kubernetes Scheduling Framework takođe razdvaja proveru uslova i ocenjivanje kandidata (The Kubernetes Authors). Simulator koristi jednostavniju verziju tog obrasca. Metoda CanFit filtrira čvorove, a konkretni raspoređivač bira jednog od preostalih kandidata.

2.3 Cilj rada

Cilj je razvoj proverljive simulacije upravljanja procesima u klasteru. Model treba da prikaže tok od prijave procesa do završnog stanja, uključujući nedostatak kapaciteta i kvar čvora, kako bi posledice svake odluke raspoređivača bile uočljive u stanju i metrikama.

Iz tog cilja izvedeni su sledeći zadaci:

1. predstaviti čvorove, procese, događaje i stanje simulacije jasnim domenskim tipovima;
2. implementirati tri raspoređivača sa različitim kriterijumima izbora;
3. održati tačne CPU i memorijske alokacije tokom svake promene stanja;
4. omogućiti determinističko izvršavanje scenarija i rad u realnom vremenu;
5. izložiti funkcije kroz scenario runner, REST API i komandni klijent;
6. prikupiti metrike koje omogućavaju poređenje raspoređivača i opterećenja;
7. proveriti ponašanje automatizovanim testovima, race detektorom i build proverom.

Sistem je obrazovni simulator. Ne pokreće udaljene izvršne fajlove, ne obavlja migraciju memorijske slike procesa i ne koordinira više kontrolnih čvorova. Ograničeni opseg omogućava ispitivanje raspoređivanja, promena stanja i oporavka u ponovljivim uslovima.

2.4 Metodologija

Metodološki okvir spaja teorijsku analizu i proveru realizovanog ponašanja. Literatura o procesima, raspoređivanju i klaster raspoređivačima daje pojmovnu osnovu, dok Go kod, testovi, scenariji i sačuvani rezultati potvrđuju tehničke tvrdnje. Planirana proširenja odvojena su od funkcija koje postoje u trenutnoj implementaciji.

Domenski model definiše stanja procesa i ograničenja resursa. Interfejs raspoređivača odvaja izbor čvora od promene stanja, dok kontroler ostaje jedina komponenta koja menja aktivno stanje klastera. Engine poziva kontroler u diskretnim taktovima ili preko vremenskog brojača. REST sloj i komandni alati koriste iste operacije da bi sva tri ulaza poštovala ista poslovna pravila.

Eksperimentalna metoda koristi JSON scenarije sa fiksnom seed vrednošću 42. Svaki scenario određuje naziv, raspoređivač, maksimalan broj taktova, čvorove, procese i opcione kvarove. JSON i CSV izvoz omogućavaju ponovno izvršavanje istog ulaza i poređenje politika prema istim metrikama. Referentne tabele preuzimaju vrednosti iz sačuvanog CSV izveštaja.

Provere su sprovedene na više nivoa. Domenski testovi proveravaju validaciju i dozvoljene prelaze stanja. Testovi raspoređivača proveravaju izbor čvora i redosled procesa. Kontroler i engine testovi pokrivaju kvarove, ograničeni rollback klasterskog stanja, konkurentne prijave, završetak bez napretka i determinističko ponavljanje. HTTP testovi koriste httptest, dok testovi izvršnih programa proveravaju izabrane CLI i konfiguracione tokove.

Sve tabele, dijagrami i grafikoni u studiji izrađeni su na osnovu realizovanog modela, izvornog koda i generisanih rezultata autora. Kada je objašnjenje zasnovano na spoljnom izvoru, citat je naveden uz odgovarajuću tvrdnju.

3. Primer/studija slučaja: implementacija simulatora i eksperimentalna analiza

3.1 Zahtevi i granica rešenja

Granica rešenja obuhvata model procesa, model klastera, odluku o raspoređivanju i merljiv rezultat simulacije. Ove četiri celine omogućavaju praćenje procesa od prijave do završetka bez izvršavanja stvarnog korisničkog programa.

Tabela 2 povezuje funkcionalne celine sa realizovanim komponentama. Relativne putanje omogućavaju proveru veze između opisa i izvornog koda.

Tabela 2: Veza funkcionalnih celina i realizovanih komponenti.

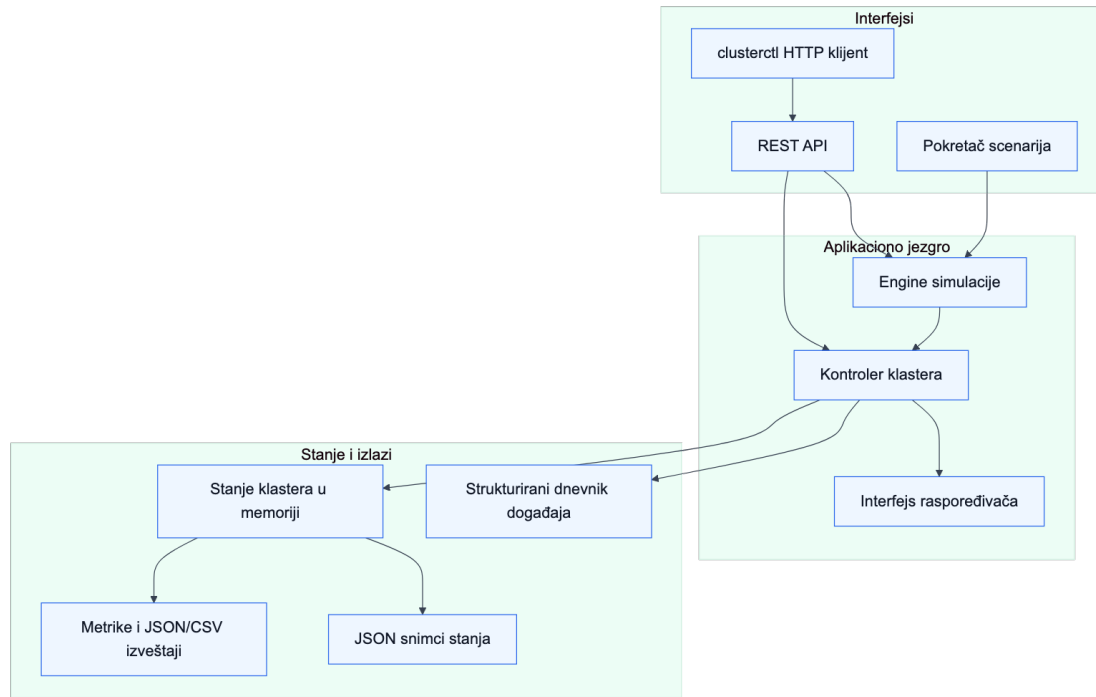
Zahtev	Realizacija	Mesto realizacije
Izvršna aplikacija	REST servis, komandni klijent i scenario runner	cmd/server, cmd/clusterctl, cmd/simulator
Model procesa	Stanja, resursi, trajanje, prioritet i politika ponovnog pokretanja	internal/domain/process.go
Model klastera	Čvorovi, statusi, kapacitet i trenutno zauzeće	internal/domain/node.go
Raspoređivanje	Round Robin, Least Loaded i Priority-Aware	internal/scheduler/
Tok simulacije	Deterministički taktovi i opciona real-time ticker	internal/engine/
Kvar i oporavak	Kvar čvora/procesa, heartbeat i ograničeno ponovno pokretanje	internal/cluster/recovery.go
Studija slučaja	Pet JSON scenarija i sedam referentnih rezultata	scenarios/, docs/results/
Provera rada	Paketni testovi, race detector, vet i build	*_test.go, Makefile, CI workflow

Aplikacija koristi samo Go standardnu biblioteku. Nema bazu podataka ni spoljni message broker. Aktivno stanje čuva se u memoriji, dok izvoz rezultata i pomoćni format snimka stanja koriste JSON ili CSV. Time su infrastrukturne zavisnosti svedene na minimum, a simulacija ostaje usmerena na upravljanje procesima.

3.2 Arhitektura aplikacije

Arhitektura ima tri ulazna interfejsa i jedno aplikaciono jezgro. cmd/simulator učitava scenario i pokreće ga do završetka. cmd/server izlaže HTTP operacije. cmd/clusterctl je klijent za isti HTTP ugovor. Sve tri putanje na kraju koriste kontroler klastera i engine.

Slika 1 prikazuje glavne komponente i smer zavisnosti. Interfejsi ne menjaju stanje direktno. Kontroler poseduje aktivno stanje, raspoređivač dobija nepromenljive snimke čvorova, a metrike se grade iz duboke kopije stanja.



Slika 1: Arhitektura aplikacije

Tabela 3 opisuje pakete koji čine završnu strukturu.

Tabela 3: Paketi završne aplikacije i njihove odgovornosti.

Paket	Odgovornost
internal/domain	Proces, čvor, scenario, događaj, stanja i validacija
internal/scheduler	Interfejs i tri politike raspoređivanja
internal/cluster	Kontroler, životni ciklus, kvarovi, scenario i invarijante
internal/engine	Determinističko i real-time izvršavanje
internal/metrics	Izračunavanje i JSON/CSV izvoz izveštaja
internal/store	Izolovana memorijska kopija i pomoćne funkcije za JSON snimak stanja
internal/transport/http	REST rute, validacija zahteva i mapiranje grešaka
internal/logging	Strukturirano JSON logovanje servera

Kontroler koristi `sync.RWMutex`. Operacije koje menjaju stanje uzimaju ekskluzivnu bravu, dok čitanje snimka i lista koristi `read` bravu. Metode za čitanje vraćaju kopije. Pozivalac zato ne može da menja mape, slice vrednosti ili pokazivače koji pripadaju aktivnom stanju.

Store paket predstavlja pomoćni sloj za izolovanu memorijsku kopiju i serijalizaciju snimka. Nije trajna baza niti obavezni runtime sloj servera. Paket proverava ugovor kopiranja i JSON format stanja koji bi u budućnosti mogao da se poveže sa fajlom ili bazom.

3.3 Domenski model procesa

Proces čuva identifikator, naziv, stanje, prioritet, CPU i memorijski zahtev, ukupno i preostalo trajanje, odabrani čvor i podatke o restartovanju. Uz to se beleže takt prijave, prvi početak, završetak i zbirno čekanje.

Tabela 4 navodi polja koja najviše utiču na izvršavanje.

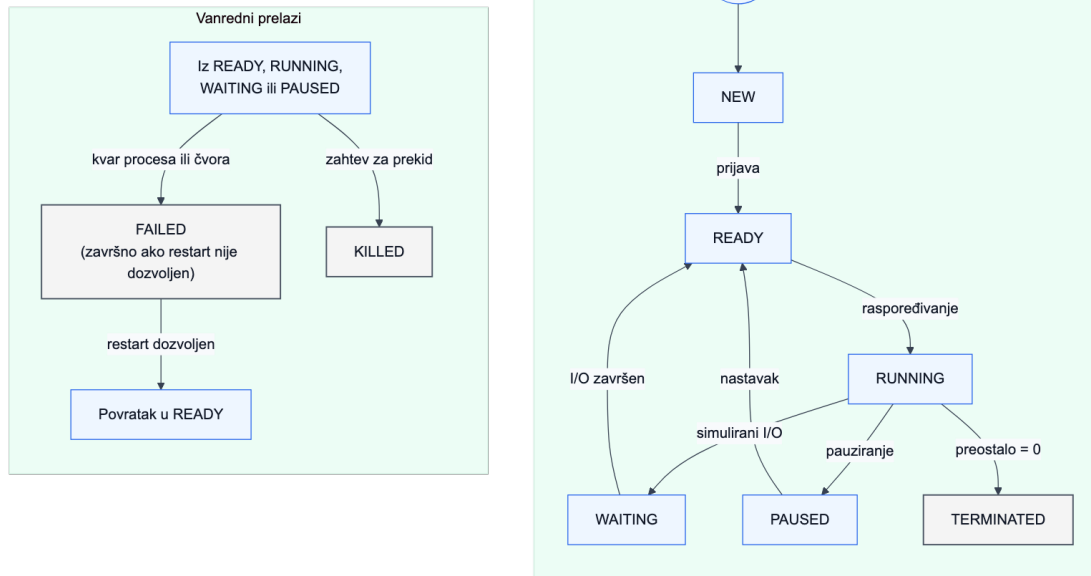
Tabela 4: Polja procesa koja utiču na izvršavanje.

Polje	Značenje u simulaciji
priority	Veća vrednost ima prednost kod Priority-Aware reda
cpuRequest	Broj CPU jedinica rezervisanih tokom RUNNING stanja
memoryRequestMB	Memorija rezervisana na čvoru
totalTicks	Trajanje jednog pokušaja procesa
remainingTicks	Broj preostalih taktova trenutnog pokušaja
restartPolicy	NEVER ili ON_FAILURE
maxRestarts	Gornja granica ponovnih pokretanja posle kvara
submitAtTick	Takt u kome proces postaje vidljiv kontroleru

Konstruktor postavlja stanje NEW, a prijava ga prevodi u READY. Raspoređivanje prelazi u RUNNING i rezerviše resurse. Simulirani I/O prelazi iz RUNNING u WAITING, pri čemu se resursi oslobađaju. Operacija wake vraća proces u READY. Pauza koristi zasebno stanje PAUSED, a resume ga takođe vraća u red spremnih procesa.

Završna stanja su TERMINATED, FAILED i KILLED. Proces postaje TERMINATED kada preostalo trajanje dođe do nule. Kvar vodi u FAILED; ako je izabrano ON_FAILURE i brojač je manji od maxRestarts, proces se odmah vraća u READY. Tada se remainingTicks postavlja na punu vrednost totalTicks. To je ponovno pokretanje od početka, a ne nastavak sa kontrolne tačke. Normalan završetak i ručni prekid ne pokreću novi pokušaj.

Slika 2 prikazuje dozvoljene prelaze. Kontroler odbija operaciju koja ne odgovara trenutnom stanju. Na primer, samo RUNNING proces može da se pauzira ili pređe u simulirano I/O čekanje.



Slika 2: Životni ciklus procesa

Validacija zahteva neprazne identifikatore i nazive, pozitivan CPU, memoriju i trajanje, kao i nenegativan broj restart pokušaja. Scenario dodatno proverava duplikate, zakazane prijave posle maxTicks i kvarove koji upućuju na nepostojeći proces ili čvor.

3.4 Politike raspoređivanja

Interfejs raspoređivača definiše metod `SelectNode(process, nodes)`. Metod dobija vrednosti `NodeSnapshot`, bez pokazivača na aktivne čvorove, čime se izbor kandidata odvaja od alokacije. Kontroler zatim proverava izabrani čvor i upisuje resurse, pa raspoređivač ne može neposredno da promeni stanje klastera.

Round Robin sortira čvorove po ID-u, pamti kursor i kruži od sledeće pozicije. Čvor koji nije ONLINE ili nema kapacitet preskače se. Kursor je zaštićen mutex-om jer isti raspoređivač može biti pozvan iz konkurentnih ulaza. Ako nijedan čvor ne odgovara, proces ostaje u READY stanju i beleži se odlaganje.

Least Loaded računa:

$$\text{score} = 0,5 * (\text{zauzeti CPU} / \text{CPU kapacitet}) + 0,5 * (\text{zauzeta memorija} / \text{memorijski kapacitet})$$

Izabran je kandidat sa najmanjom ocenom pre smeštanja procesa. Ako dve ocene imaju istu vrednost, bira se leksikografski manji ID. Ovo pravilo uklanja zavisnost od redosleda iteracije kroz Go mapu.

Priority-Aware menja redosled READY procesa, a za izbor čvora koristi Least Loaded. Efektivni prioritet je osnovni prioritet uvećan za jedan poen na svakih pet taktova provedenih u redu. Veća vrednost ide prva. Kod izjednačenja prednost ima ranija prijava, a zatim manji ID. Aging smanjuje rizik da proces niskog prioriteta čeka bez granice.

Tabela 5 daje sažeto poređenje.

Tabela 5: Poređenje realizovanih politika raspoređivanja.

Raspoređivač	Redosled procesa	Izbor čvora	Tipična namena
Round Robin	prijava, zatim ID	kružni izbor dostupnih čvorova	jednostavna i ponovljiva raspodela
Least Loaded	prijava, zatim ID	najmanja kombinovana CPU/ memorijska popunjenost	ravnomernije opterećenje
Priority-Aware	efektivni prioritet, prijava, ID	Least Loaded	hitni procesi uz aging

Round Robin ne garantuje najbolji balans kada čvorovi imaju različite kapacitete. Least Loaded ne uzima u obzir očekivano trajanje procesa. Priority-Aware može da odloži procese niskog prioriteta dok je kapacitet pun, iako aging postepeno popravlja njihov položaj. Zbog tih razlika rezultati se porede nad istim ulazom.

3.5 Jedan takt simulacije

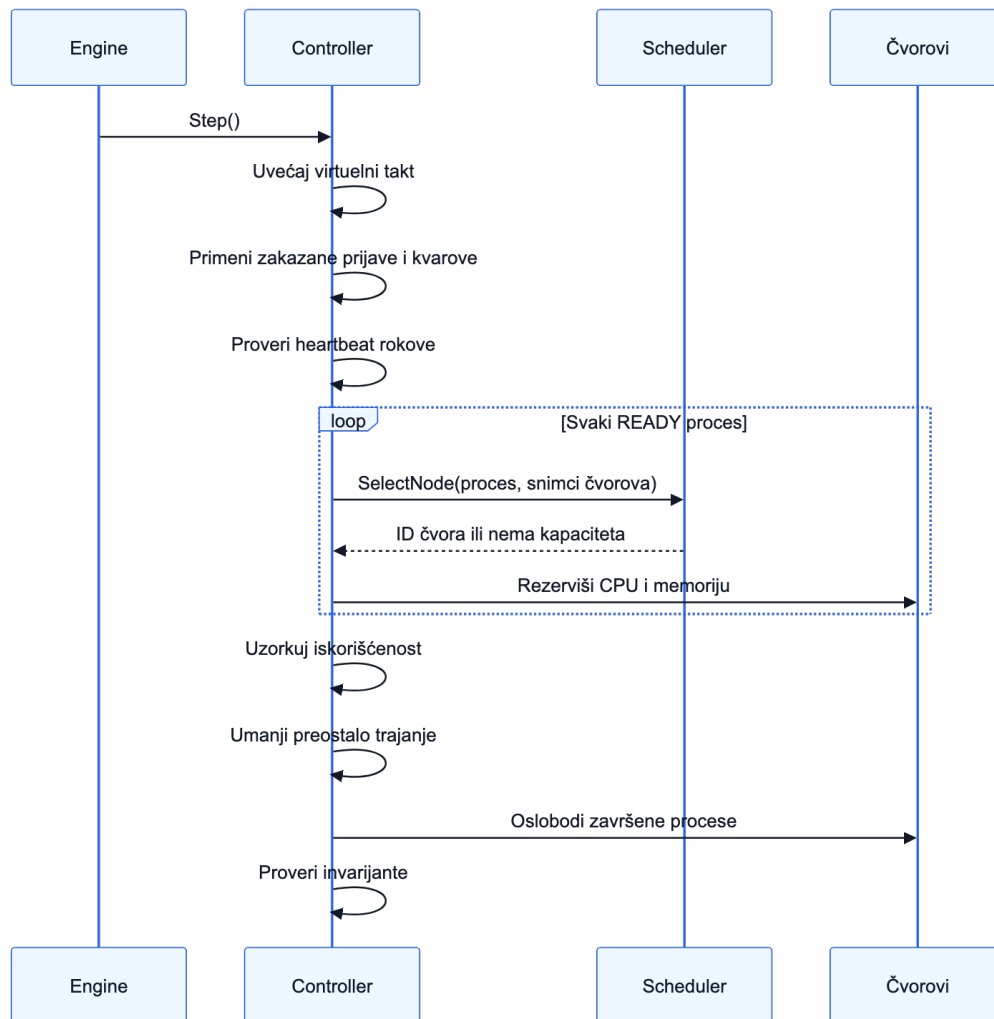
Engine može da pozove jedan ili više determinističkih koraka. Može i da pokrene goroutine sa `time.Ticker` petljom za rad u realnom vremenu. `Pause` i `Close` otkazuju petlju preko `context.Context` i čekaju njen završetak. Upotreba konteksta prati standardni Go obrazac za prenos otkazivanja između goroutine-a (Ajmani, 2014).

Kontroler u svakom koraku radi sledeće:

1. čuva kopiju klusterskog stanja i zakazanih stavki potrebnu za rollback;
2. uvećava virtuelni takt;
3. prijavljuje procese i primenjuje kvarove zakazane za taj takt;
4. proverava heartbeat rokove;
5. sortira i raspoređuje READY procese;
6. uzorkuje CPU, memoriju i odstupanje opterećenja;
7. umanjuje `remainingTicks` aktivnim procesima;
8. završava procese čije je trajanje došlo do nule;
9. određuje da li je simulacija završena;
10. proverava invarijante.

Rollback takta obuhvata klustersko stanje, broj događaja, sekvencu i preostale zakazane stavke. Procesi, čvorovi i zakazane definicije kopiraju se pre izmene. Dnevnik događaja se tokom zaključanog takta samo dopunjava, pa se pri grešci vraća na prethodnu dužinu. Interno promenljivo stanje raspoređivača nije deo ovog snimka. Round Robin kursor zato može ostati pomeren ako se greška javi posle izbora čvora. Mehanizam sprečava delimično izmenjene alokacije u klusterskom stanju, ali ne daje potpunu transakcionu semantiku svim komponentama.

Slika 3 prikazuje isti redosled i granicu između engine-a, kontrolera, raspoređivača, stanja i metrika.



Slika 3: Sekvenca jednog takta

Kriterijum završetka određuje kada dalje izvršavanje ne može da promeni ishod scenarija. Simulacija se završava kada su svi procesi u završnom stanju, kada se dostigne maxTicks ili kada napredak nije moguć. Rana provera primenjuje se tek kada nema aktivnog procesa, buduće prijave ni zakazanog kvara. Ako nijedan READY proces ne može da stane na ONLINE čvor, razlog je NO_ONLINE_CAPACITY. Ovaj razlog obuhvata i nominalno dovoljno veliki čvor u stanju DRAINING, OFFLINE ili FAILED. Ako su preostali procesi samo WAITING ili PAUSED, rezultat je EXTERNALLY_BLOCKED, jer nastavak zahteva spoljnu komandu wake ili resume. Kriterijum sprečava čekanje do veštačke granice kada scenario više nema događaj koji može da omogućiti napredak.

3.6 Kvar čvora i ponovno raspoređivanje

Kvar može biti ručno izazvan preko API-ja, zakazan u scenariju ili otkriven prekoračenjem heartbeat roka. Sva tri ulaza koriste isti failNodeLocked tok. Čvor prelazi u FAILED, njegovi brojači alokacije vraćaju se na nulu, a lista procesa se obrađuje po sortiranom ID redosledu.

Svaki pogođeni proces prelazi u FAILED. Ako CanRestart vrati tačno, brojač restartovanja se uvećava, trajanje se vraća na početnu vrednost i proces prelazi u READY. Sledeće raspoređivanje može da ga smesti na drugi ONLINE čvor. Ako nema dozvoljen pokušaj, proces ostaje u završnom FAILED stanju.

Tabela 6 daje potpunu semantiku politika za konačne poslove.

Tabela 6: Semantika politika ponovnog pokretanja prema završnom događaju.

Događaj	NEVER	ON_FAILURE
Normalan završetak	TERMINATED	TERMINATED
Kvar procesa	FAILED	Novi pokušaj dok je restartCount < maxRestarts
Kvar čvora	FAILED	Novi pokušaj dok je restartCount < maxRestarts
Ručni prekid	KILLED	KILLED

Oporavak čvora je odvojen. RecoverNode prihvata FAILED ili OFFLINE, kratko postavlja RECOVERING, zatim ONLINE i osvežava poslednji heartbeat. Oba prelaza nastaju pod istom ekskluzivnom bravom, pa spoljni snimak vidi završno ONLINE stanje, dok se međustanje beleži događajem. RECOVERING zato nije stanje u kome čvor čeka više taktova. Ranije alokacije se ne vraćaju.

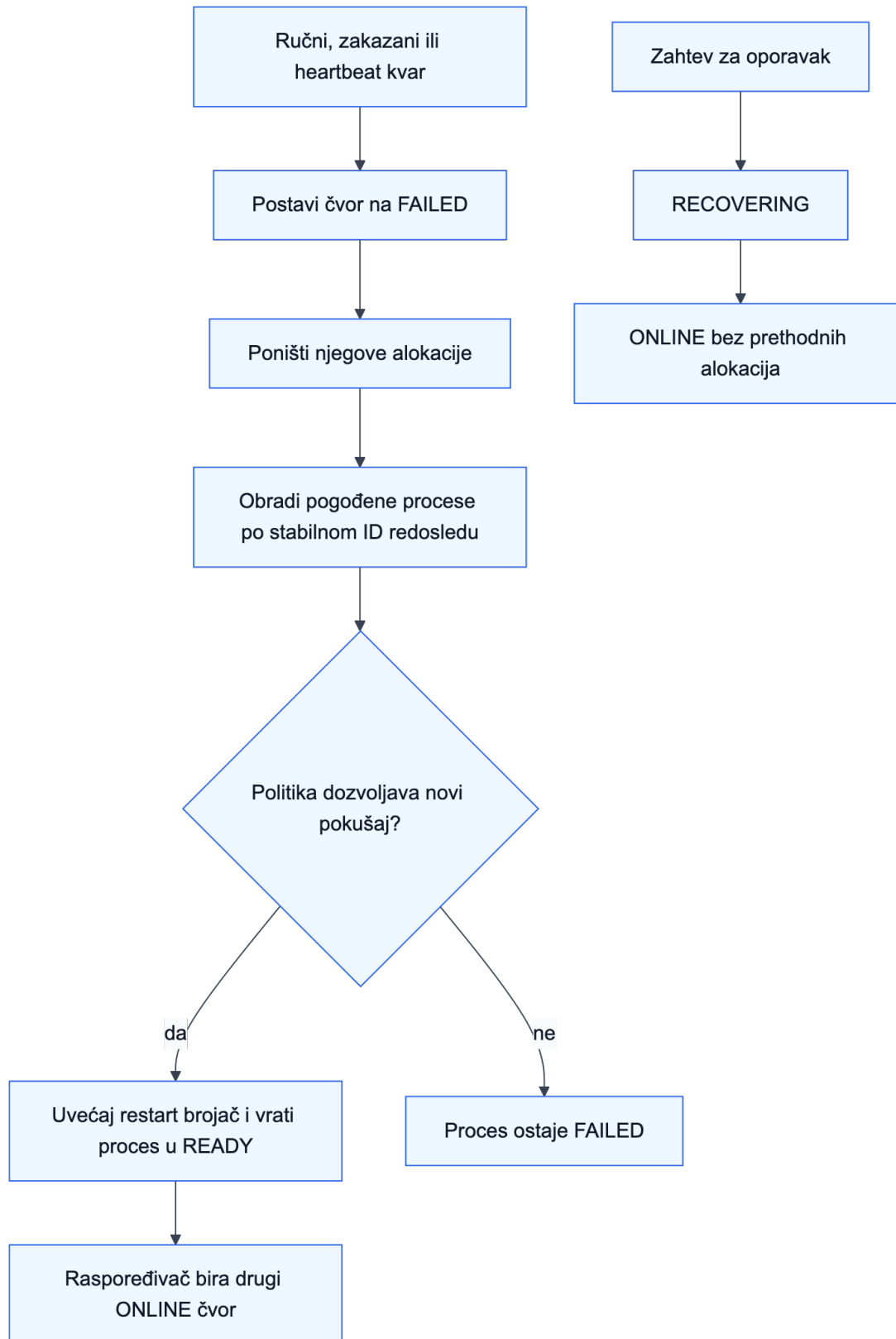
Tabela 7 navodi invarijante koje se proveravaju posle svakog uspešnog takta.

Tabela 7: Resursne i referencijalne invarijante klastera.

Invarijanta	Provera
Kapacitet čvora	zauzeti CPU i memorija su između nule i kapaciteta
Tačni brojači	zbir zahteva aktivnih procesa odgovara brojačima čvora
Jedinstvena alokacija	svaki RUNNING proces pripada tačno jednom čvoru
Saglasan NodeID	proces i lista čvora navode isti odnos
Bez zaostale alokacije	proces van RUNNING stanja nema NodeID
Bez nepoznatih procesa	svaki ID u listi čvora postoji u registru procesa

Invarijante se proveravaju tokom izvršavanja i u testovima. Njihova namena je da odmah odbiju stanje sa prekomerno zauzetim čvorom ili procesom koji je istovremeno raspoređen na dve lokacije.

Slika 4 prikazuje obradu kvara, odluku o ponovnom pokretanju i naknadni oporavak čvora.



Slika 4: Obrada kvara i oporavka

3.7 REST API i komandni alati

REST interfejs preslikava domenske operacije na HTTP resurse pod prefiksom /api/v1. Rute su podeljene na čvorove, procese, simulaciju, raspoređivač, događaje i izveštaje, dok /healthz služi

proveri dostupnosti servisa. Komandni klijent i drugi HTTP potrošači zato koriste ista pravila kontrolera.

Tabela 8 prikazuje glavne grupe operacija.

Tabela 8: Grupe operacija REST interfejsa.

Resurs	Primeri operacija
Čvorovi	registracija, lista, status, kvar, oporavak, heartbeat i brisanje
Procesi	prijava, lista, pauza, nastavak, I/O čekanje, buđenje, kvar i prekid
Simulacija	start, pauza, koraci, reset, status i učitavanje scenarija
Raspoređivač	pregled dostupnih politika i promena aktivne politike
Izveštaji	trenutne metrike, zbirni pregled i JSON/CSV izvoz

JSON dekodirer odbija nepoznata polja. Telo zahteva ograničeno je na 1 MiB. Domenske greške se prevode u HTTP statute 400, 404, 409 ili 422, dok neočekivana greška vraća 500. Standardna prijava procesa bez slobodnog kapaciteta ipak vraća 201 i ostavlja proces u stanju READY; raspoređivanje se odlaže do oslobađanja resursa. Mapiranje 422 postoji za domensku grešku nedostatka kapaciteta, ali prikazani javni tok i testovi ga ne demonstriraju. Server postavlja rokove za čitanje zaglavlja, čitanje tela, pisanje odgovora i neaktivne konekcije.

Najčešći statusi imaju sledeće značenje:

Status	Ugovor
200	Uspešno čitanje ili promena postojećeg resursa
201	Kreiran čvor ili proces, odnosno učitani scenarij
204	Uspešno brisanje bez tela odgovora
400	Neispravan JSON ili ulazna vrednost
404	Čvor ili proces ne postoji
409	Duplikat ili nedozvoljen prelaz stanja

Prijava procesa vraća 201 i stanje READY. Reprezentativni zahtev i skraćeni odgovor su:

```
{
  "id": "p1",
  "name": "image-worker",
  "priority": 10,
  "cpuRequest": 2,
  "memoryRequestMB": 512,
  "totalTicks": 20,
  "restartPolicy": "ON_FAILURE",
  "maxRestarts": 2
}

{
  "id": "p1",
  "state": "READY",
  "remainingTicks": 20,
  "restartPolicy": "ON_FAILURE",
  "restartCount": 0,
  "maxRestarts": 2,
  "waitingTicks": 0
}
```

clusterctl koristi isti API. Globalna opcija --server određuje adresu, a podkomande prate domenske operacije. Primer prijave čvora i procesa izgleda ovako:

```
go run ./cmd/clusterctl node add \
  --id node-1 --cpu 4 --memory 4096
```

```
go run ./cmd/clusterctl process submit \
  --id p1 --cpu 1 --memory 256 \
  --ticks 5 --priority 10
```

```
go run ./cmd/clusterctl simulation step --ticks 5
go run ./cmd/clusterctl report show
```

Samostalni simulator ne zahteva pokrenut server:

```
go run ./cmd/simulator \
  -scenario scenarios/node-failure.json \
  -format json
```

Opcija -scheduler menja raspoređivač iz scenarija, a -seed menja nenultu seed vrednost. Putanja izlaznog JSON ili CSV fajla zadaje se opcijom -output. Dekoder scenarija ograničava ulaz na 4 MiB i odbija nepoznata polja.

3.8 Konfiguracija i isporuka

Konfiguracija određuje operativne granice servera bez promene izvornog koda. Server je čita iz promenljivih okruženja. Podrazumevani port je 8080, takt traje 500 ms, automatsko otkrivanje heartbeat prekoračenja isključeno je vrednošću 0, a početni raspoređivač je Least Loaded. Postavljene granice iznose 100 čvorova i 10.000 procesa.

Tabela 9 prikazuje dostupne promenljive.

Tabela 9: Promenljive okruženja i podrazumevane vrednosti.

Promenljiva	Podrazumevana vrednost	Uloga
APP_PORT	8080	HTTP port
LOG_LEVEL	info	nivo JSON logova
TICK_DURATION_MS	500	real-time interval takta
HEARTBEAT_TIMEOUT_TICKS	0	broj propuštenih taktova do kvara
DEFAULT_SCHEDULER	least-loaded	početna politika
MAX_NODES	100	granica registracije čvorova
MAX_PROCESSES	10000	granica prijave procesa
RANDOM_SEED	42	seed početnog stanja

Dockerfile koristi višefazni build. Prva faza kompajlira statički server binarni fajl, dok završna distroless slika radi kao nonroot korisnik. Docker Compose prosleđuje osnovnu konfiguraciju i mapira port 8080. GitHub Actions workflow proverava format, vet, testove sa coverage izveštajem, race detector, sve binarne fajlove i container build.

3.9 Metrike

Metrika je brojčana karakteristika ponašanja simulacije. Izveštaj se gradi iz snimka klastera i posebno broji prijavljene, pokrenute i nikada pokrenute procese, kao i procese po stanju. Beleže se restartovanja, odlaganja raspoređivanja, kvarovi čvorova i ponovna raspoređivanja. Vremenske metrike obuhvataju čekanje, vreme obrta, propusnost i ukupan broj taktova, čime se politike mogu porediti nad istim scenarijem.

CPU iskorišćenost je odnos ukupno zauzetog CPU kapaciteta i zbira kapaciteta svih registrovanih čvorova, bez filtriranja prema statusu. Memorijska iskorišćenost računa se na isti način. FAILED, OFFLINE i DRAINING čvorovi zato ostaju u imeniocu iako ne prihvataju novo opterećenje. Obe vrednosti uzorkuju se jednom po taktu, posle raspoređivanja i pre izvršavanja procesa.

Balans opterećenja koristi standardnu devijaciju ocena svih registrovanih čvorova. Ocena jednog čvora je ista ponderisana kombinacija koju koristi Least Loaded, ali skup za metriku uključuje i čvorove koji nisu ONLINE. Manja standardna devijacija znači da su čvorovi u tom skupu u proseku sličnije opterećeni. Metrika ne tvrdi da je takav raspored uvek bolji; ona meri jednu tačno definisanu osobinu.

Uspešnost je odnos TERMINATED procesa prema svim prijavljenim procesima. Propusnost je broj završenih procesa podeljen brojem taktova. Vreme obrta koristi razliku između završnog i prijavnog takta.

Čekanje se objavljuje kroz dva ugovora. averageWaitingTicksStarted deli zbir READY intervala brojem procesa koji su makar jednom počeli izvršavanje. averageWaitingTicksAllSubmitted uključuje svaki prijavljeni proces i vreme posmatrano do trenutka izveštaja, završetka ili prekida. Polje neverStartedProcesses sprečava da neizvodljiv proces nestane iz proseka i tako veštački poboljša rezultat raspoređivača. U sedam referentnih pokretanja svi procesi su počeli izvršavanje, pa dve prosečne vrednosti imaju isti broj.

3.10 Eksperimentalni scenariji

Eksperimentalni scenario određuje čvorove, procese, politiku raspoređivanja i događaje jednog ponovljivog pokretanja. Skup obuhvata pet scenario fajlova. Balansirani i heterogeni ulaz pokreću se po dva puta uz promenu opcije -scheduler, dok preopterećeni, prioritetni i scenario kvara koriste politiku zapisanu u fajlu. Tako nastaje sedam redova rezultata, pri čemu svako neposredno poređenje dve politike zadržava isti ulaz.

Tabela 10 opisuje ulaze.

Tabela 10: Referentni eksperimentalni scenariji.

Scenario	Čvorovi	Procesi	Raspoređivač	Posebna osobina
balanced.json	5	12	Round Robin ili Least Loaded	jednaki čvorovi i umereno opterećenje
heterogeneous.json	3	6	Least Loaded	različiti kapaciteti čvorova
overload.json	2	8	Least Loaded	više zahteva od trenutnog kapaciteta
priority-workload.json	2	7	Priority-Aware	urgentni proces dolazi u taktu 2
node-failure.json	3	5	Least Loaded	zakazan kvar čvora u taktu 4

Svi referentni ulazi koriste seed 42 i deterministički engine. Rezultati se dobijaju komandom:

```
go run ./cmd/simulator \
  -scenario scenarios/<naziv>.json \
  -format json
```

Za heterogeni scenario poređenje se ponavlja ovako:

```
go run ./cmd/simulator \
  -scenario scenarios/heterogeneous.json \
  -scheduler round-robin
```

```
go run ./cmd/simulator \
  -scenario scenarios/heterogeneous.json \
  -scheduler least-loaded
```

3.11 Rezultati

Tabele 11a i 11b prenose vrednosti iz referentnog summary.csv izveštaja. Širok skup kolona podeljen je radi čitljivosti, a procenti su izračunati iz decimalnih vrednosti koje daje aplikacija. U svih sedam pokretanja važi neverStartedProcesses = 0, što znači da nijedan prijavljeni proces nije ostao bez prvog izvršavanja.

Tabela 11a: Vremenske metrike i uspešnost referentnih pokretanja.

Pokretanje	Taktovi	Čekanje pokrenutih	Propusnost	Uspeh
Balansirano, Round Robin	9	1,00	1,333	100%
Balansirano, Least Loaded	9	1,00	1,333	100%
Heterogeno, Round Robin	22	3,00	0,273	100%
Heterogeno, Least Loaded	22	3,00	0,273	100%
Veliko opterećenje	39	11,25	0,205	100%
Prioriteti	14	3,43	0,500	100%
Kvar čvora	16	1,80	0,313	100%

Tabela 11b: Iskorišćenost, balans, odlaganja i ponovna pokretanja u referentnim pokretanjima.

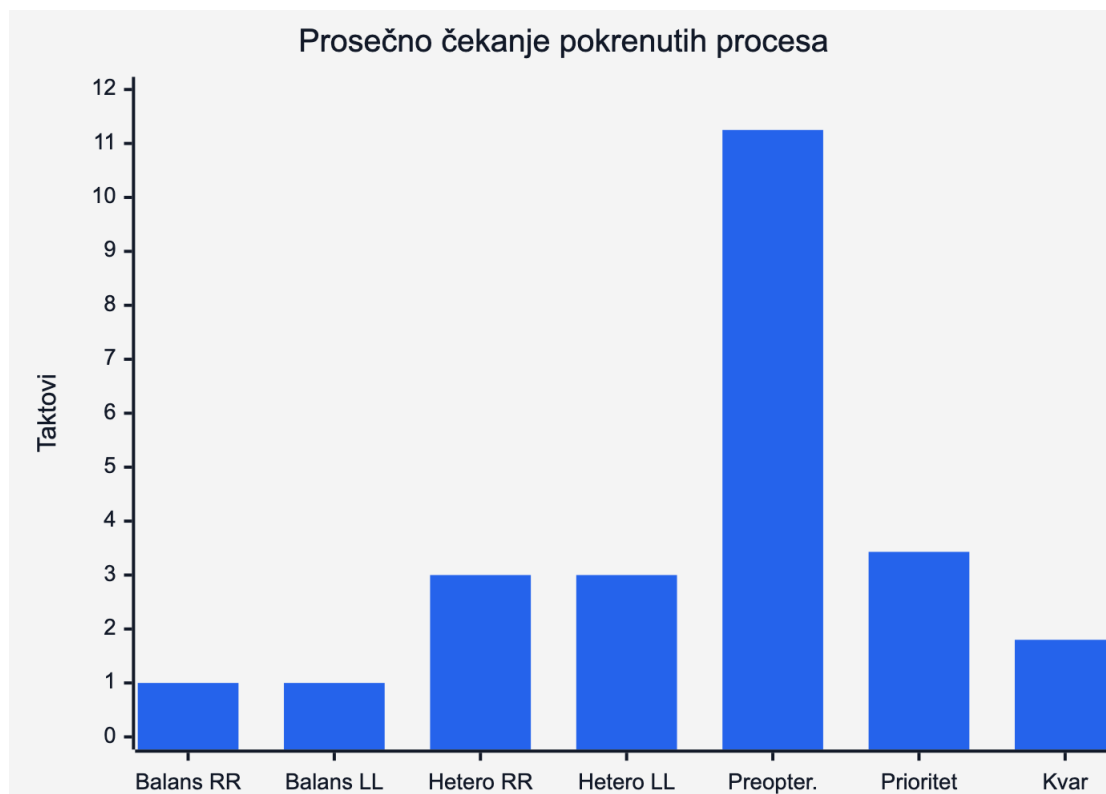
Opterećenje	Prosečan CPU	Load σ	Odlaganja	Ponovna pokretanja
Balansirano, Round Robin	52,22%	0,1393	0	0
Balansirano, Least Loaded	52,22%	0,0987	0	0
Heterogeno, Round Robin	49,68%	0,2417	12	0
Heterogeno, Least Loaded	49,68%	0,2452	12	0
Veliko opterećenje	89,10%	0,0962	82	0
Prioriteti	75,00%	0,0469	18	0
Kvar čvora	54,17%	0,2368	4	2

Slika 5 prikazuje ukupan broj taktova. Preopterećeni scenario traje 39 taktova, dok oba balansirana pokretanja završavaju za 9. Oznake RR i LL znače Round Robin i Least Loaded, a Balans i Hetero označavaju balansirani i heterogeni ulaz.



Slika 5: Trajanje referentnih scenarija; RR je Round Robin, LL je Least Loaded

Slika 6 prikazuje prosečno čekanje pokrenutih procesa. Rast sa 1,00 na 11,25 taktova prati ograničenje kapaciteta u preopterećenom scenariju. Pošto su svi procesi u ovim pokretanjima započeti, prosek za sve prijavljene procese ima iste vrednosti.



Slika 6: Prosečno čekanje pokrenutih procesa; Balans označava balansirani, a Hetero heterogeni ulaz

3.12 Tumačenje balansiranog i heterogenog opterećenja

Balansirano opterećenje predstavlja kontrolni slučaj sa jednakim kapacitetima i prioritetima. Pet čvorova ima po 4 CPU jedinice i 4096 MB memorije, a 12 procesa ima iste prioritete. Round Robin i Least Loaded završavaju svih 12 procesa za 9 taktova. Prosečno čekanje je 1 takt, prosečno vreme obrta 6,17 taktova, a propusnost 1,333 procesa po taktu.

Razlika se javlja u raspodeli opterećenja. Least Loaded smanjuje prosečnu standardnu devijaciju sa 0,1393 na 0,0987, odnosno za približno 29%. Ukupno vreme ostaje isto jer oba rasporeda imaju dovoljno kapaciteta. Rezultat pokazuje da kriterijum Least Loaded ravnomernije raspoređuje ovaj ulaz, ali u datim uslovima ne skraćuje njegovo izvršavanje.

Heterogeni scenario ima tri čvora različitih kapaciteta i šest procesa. Oba raspoređivača završavaju za 22 takta sa prosečnim čekanjem od 3 takta i 12 odlaganja. Najveće čekanje je 13 taktova. Ograničenja kapaciteta izbacuju premale čvorove za najveće procese, pa razlika u heuristici ne menja tok dovoljno da utiče na završne vremenske metrike.

Standardna devijacija opterećenja u ovom slučaju neznatno je manja kod Round Robin raspoređivača, 0,2417 naspram 0,2452. Least Loaded zato nije bolji prema svakoj metrici. Njegova ocena koristi popunjenost pre smeštanja i ne uključuje očekivano trajanje posla. U ovoj heterogenoj raspodeli stabilan kružni izbor daje malo manju zbirnu meru odstupanja.

3.13 Tumačenje preopterećenja i prioriteta

Preopterećeni scenario ima dva čvora i osam procesa. Svi procesi završavaju, ali ne mogu svi da krenu u prvom taktu. CPU je prosečno zauzet 89,10%, memorija 79,49%, a kontroler beleži 82 neuspešna pokušaja raspoređivanja. Prosečno čekanje raste na 11,25, maksimalno na 31, a vreme obrta na 23,88 taktova.

Odlaganje raspoređivanja označava neuspešan pokušaj smeštanja zbog trenutnog nedostatka kapaciteta. Proces tada ostaje READY, a završetak aktivnog procesa kasnije oslobađa resurse i omogućava napredovanje reda. Ovaj mehanizam čuva resursne invarijante umesto da prekoračenjem kapaciteta skрати rezultat.

Prioritetni scenario koristi dva čvora i sedam procesa. Dva procesa imaju prioritet 10, dva prioritet 5, dva prioritet 1, a proces urgent sa prioritetom 20 ulazi u taktu 2. Priority-Aware sortira red prema efektivnom prioritetu, dok aging postepeno povećava vrednost procesa koji čekaju.

Scenario završava za 14 taktova, sa prosečnim čekanjem 3,43 i 18 odlaganja. Svi procesi završavaju. Prioritet menja redosled razmatranja, ali ne povećava raspoloživi kapacitet. Kada su oba čvora puna, proces urgent čeka oslobađanje potrebne dve CPU jedinice.

3.14 Tumačenje kvara čvora

Scenario kvara ima tri jednaka čvora sa po 2 CPU jedinice i 2048 MB memorije. Pet procesa koristi politiku ON_FAILURE i dozvoljava najviše dva restarta. U taktu 4 zakazan je kvar node-1.

Kvar pogađa dva aktivna procesa. Oba prelaze u FAILED, koriste po jedan ponovni pokušaj, vraćaju se u READY i kasnije dobijaju zdrave čvorove. Izveštaj zato beleži dva ponovna pokretanja i dva ponovna raspoređivanja. Ne beleži neuspešne završne procese.

Ukupno trajanje je 16 taktova. Prosečno čekanje iznosi 1,80, prosečno vreme obrta 11,20, a zabeležena su četiri odlaganja raspoređivanja. Propusnost je 0,3125 procesa po taktu. Završni uspeh od 100% potvrđuje da politika ponovnog pokretanja radi u granicama scenarija.

Rezultat ne predstavlja live migraciju. Pogođeni procesi počinju novi pokušaj sa punim totalTicks. Broj reschedulings označava da je proces koji je već bio pokrenut ponovo smešten. Dokument koristi izraz „ponovno raspoređivanje” da bi se ta razlika zadržala.

3.15 Ponovljivost i funkcionalna provera

Za fiksni scenario raspoređivač dobija stabilno sortirane podatke. Pri istom prijavnom taktu procesi se sortiraju po ID-u, a isti kriterijum važi i za čvorove. Least Loaded koristi ID kao završni kriterijum. Zakazani događaji vezani su za virtuelni takt. Zbog toga isti ulaz i ista politika daju isti izveštaj.

Ponovljivost je proverena scenarijem kvara čvora, poređenjem Round Robin i Least Loaded politika nad istim heterogenim ulazom i pozivom REST servisa kroz clusterctl. Scenario kvara daje nodeFailures: 1, restarts: 2 i reschedulings: 2, dok poređenje raspoređivača zadržava isti JSON ulaz i menja samo izabranu politiku.

Komanda korišćena za jedno od ponovljivih poređenja je:

```
go run ./cmd/simulator \  
-scenario scenarios/heterogeneous.json \  
-scheduler round-robin \  
-format csv
```

Za drugi prolaz menja se samo -scheduler least-loaded. Isto pravilo važi za seed, iako postojeći scenariji nemaju slučajnu generaciju kvarova. Seed je deo izveštaja i ostavlja prostor za buduće proširenje bez promene izlaznog ugovora.

Mašinski čitljiv manifest reproduktivnosti beleži osnovni Git commit, zbirni SHA-256 definisanog skupa implementacionih i scenario fajlova, verziju Go alata, hash svakog scenarija i oba zbirna izveštaja. Ne obuhvata kompletno predajno stablo, DOCX, PDF ni README fajlove i nije nezavisna potvrda izvršenja komandi. Generator u repozitorijumu čita commit iz Git metapodataka, dok u predajnom paketu koristi SOURCE_COMMIT.txt, jer arhiva namerno nema .git direktorijum. Manifest zato služi za povezivanje jasno navedenog podskupa ulaza, koda i rezultata, uz navedena ograničenja.

3.16 Testiranje

Automatizovani testovi obuhvataju izvršne programe i interne pakete. Test funkcije koriste table-driven slučajeve tamo gde ista pravila imaju više ulaza.

Tabela 12 izdvaja ponašanja koja nose najveći rizik.

Tabela 12: Automatizovane provere najrizičnijih ponašanja.

Oblast	Provereno ponašanje
Domen	validacija procesa, čvora i scenarija; dozvoljeni prelazi
Raspoređivač	kružni kursor, preskakanje punog čvora, ocena opterećenja i aging
Kontroler	alokacija, oslobađanje, pauza, I/O, prekid i konkurentne prijave
Kvarovi	kvar čvora/procesa, granica ponovnih pokretanja, heartbeat timeout i oporavak
Scenario	učitavanje bez delimičnog stanja, zakazane prijave/kvarovi, maxTicks, nedostupni čvorovi i spolja blokirana stanja
Engine	determinističko ponavljanje, real-time završetak, pause i close
Metrike	pokrenuti i nikada pokrenuti procesi, dva proseka čekanja, obrt, propusnost, balans i JSON/CSV red
HTTP	kreiranje čvora i procesa, duplikat, nedostajući proces, neispravan JSON, korak simulacije, metrike i CSV izvoz
CLI	obrada izabranih argumenata, izlaz simulatora i validacija konfiguracije servera
Store	duboke kopije i JSON round-trip snimka

Race detector je relevantan zbog RWMutex zaštite kontrolera, mutex-a Round Robin kursora i goroutine petlje engine-a. Testovi uključuju konkurentne prijave, ali nema posebnog testa paralelnih čitanja stanja. HTTP scenario load, kompletna matrica ruta i gašenje celog servera pod signalom takođe nisu pokriveni prikazanim testovima. Postojeće provere obuhvataju izvršene putanje, ali ne dokazuju odsustvo svake moguće konkurentne greške. Logging paket ostaje mali otvoreni rizik jer nema zaseban direktan test.

3.17 Korektnost i složenost

Korektnost je definisana pravilima modela. Proces ne sme da bude RUNNING bez čvora. Čvor ne sme da navede proces koji ne postoji. CPU i memorijski brojači moraju odgovarati zbiru aktivnih procesa. Oslobađanje resursa mora da ukloni proces iz liste čvora. Ova pravila proverava ValidateInvariants posle svakog takta.

Neka je P broj procesa, N broj čvorova, Q broj preostalih zakazanih definicija, a T broj izvršenih taktova. Pre izmene kontroler duboko kopira mape procesa i čvorova, njihove liste alokacija i zakazane definicije. Taj deo košta $O(P + N + Q)$ po taktu. Dnevnik događaja se ne kopira; čuva se samo prethodna dužina, što je $O(1)$.

Kontroler zatim prolazi kroz procese da pronade READY i RUNNING skup. Sortiranje reda košta $O(P \log P)$. Izbor čvora za svaki spreman proces u najgorem slučaju pregleda sve čvorove, pa košta $O(PN)$. Provera invarijanti ponovo prolazi kroz čvorove i njihove aktivne procese. Gornja procena jednog takta je zato $O(P \log P + PN + P + N + Q)$, a celog scenarija zbir tog troška kroz T taktova. Za zadate scenarije sa najviše 12 procesa taj trošak je mali. Procena nije zamena za merenje na konfigurisanoj granici od 10.000 procesa.

Trajni memorijski trošak je $O(P + N + E + Q)$, gde je E broj događaja. Rollback klasterskog stanja privremeno zahteva još $O(P + N + Q)$. Stanje raspoređivača nije uključeno u tu procenu niti u snimak za vraćanje. Dnevnik događaja raste tokom rada i nema runtime limit ni rotaciju. To je prihvatljivo za kratke scenarije, ali nije odgovarajuće za dugotrajn servis.

3.18 Bezbednosne i operativne granice

Server nema autentifikaciju ni autorizaciju. Svaki klijent koji može da pristupi portu može da menja stanje simulacije. Sistem zato treba pokretati lokalno ili u izolovanoj mreži. Nije predviđen za izlaganje na javni internet.

Ulazni JSON ima ograničenje veličine i odbija nepoznata polja. HTTP server koristi rokove i ograničava zaglavlja. Docker proces radi bez root privilegija. Ove mere smanjuju deo tehničkog rizika, ali ne menjaju obrazovnu namenu sistema.

Procesi su podaci. Simulator ne izvršava komandu iz polja `name` ili `id`, pa scenario ne može direktno da pokrene korisnički kod. CSV i JSON izvoz sadrže izračunate metrike, bez tajni ili podataka o autentifikaciji.

3.19 Ograničenja i dalji rad

Apstrakcija procesa određuje osnovno ograničenje rezultata. CPU i memorija su rezervacije u modelu, a ne izmerena potrošnja, dok je trajanje izraženo brojem taktova. Rezultati zato porede algoritme unutar simulatora i ne predstavljaju performanse stvarnog klastera.

Aktuelni ugovor je nepreemptivan i ne prihvata vremenski kvant. Metoda `executeRunningLocked` umanjuje trajanje svakog aktivnog procesa u svakom taktu. Buduće preemptivno proširenje zahtevalo bi kvantni brojač, događaj preempcije, oslobađanje resursa i povratak u `READY`, kao i posebne testove pravednosti i čekanja.

Ponovno pokretanje ne koristi kontrolnu tačku. Kvar vraća `remainingTicks` na `totalTicks`. Za model nastavka bilo bi potrebno čuvati napredak i definisati da li se deo rada ponavlja. Za stvarni proces bili bi potrebni memorijska slika, otvoreni resursi i protokol oporavka.

Heartbeat je izražen u virtuelnim taktovima. Ne modeluje kašnjenje mreže, privremenu particiju ili lažno pozitivan kvar. Sistem ima jedan kontroler i nema konsenzus između replika. Distribuirani log i izbor vođe predstavljaju zaseban problem, kakav obrađuje Raft literatura (Ongaro i Ousterhout, 2014).

Simulirani I/O koristi eksplicitne operacije `wait` i `wake`. Trajanje I/O zahteva, red uređaja i probablistička mrežna latencija nisu modelovani. Njihovo buduće uvođenje zahtevalo bi zaseban sloj zakazanih događaja i nove eksperimentalne rezultate.

Least Loaded koristi jednake težine CPU-a i memorije. Konfigurabilne težine i ocena posle zamišljenog smeštanja procesa dale bi precizniji kriterijum. Raspoređivač takođe ne poznaje oznake čvorova, afinitet, zone ili posebne uređaje. Kubernetes dokumentacija pokazuje kako oznake i pravila utiču na izbor čvora (The Kubernetes Authors).

Pomoćne funkcije za snimak stanja nisu trajno skladište servera. Povezivanje sa fajlom ili bazom zahtevalo bi verzionisanje formata, proveru invarijanti pri učitavanju i odluku kada se snimak pravi. Trenutni `ReadSnapshot` proverava JSON oblik, ali ne izvršava punu domensku validaciju obnovljenog klastera.

Eksperimentalni skup može da se proširi. Korisni naredni slučajevi su postepeni gubitak kapaciteta, više kvarova, procesni kvar bez kvara čvora i mešovita CPU/memorijska opterećenja. Kod scenarija sa slučajno generisanim događajima rezultat treba ponoviti nad više seed vrednosti i prijaviti srednju vrednost, standardnu devijaciju i interval pouzdanosti.

4. Zaključak

Razvijena Go aplikacija predstavlja upravljanje procesima u klasteru kroz proverljiv model. Sistem ima osam stanja procesa, pet statusa čvora, tri raspoređivača, nepreemptivni deterministički engine, REST API, komandni klijent i izvoz metrika. Kvarovi čvora i procesa koriste isti kontrolni tok, a resursne invarijante proveravaju se posle svakog takta.

Rezultati pokazuju razliku između heuristike i kapaciteta. Least Loaded je u balansiranom scenariju smanjio odstupanje opterećenja za približno 29%, bez promene ukupnog vremena. U heterogenom scenariju oba poređena raspoređivača dala su iste vremenske metrike jer su ograničenja kapaciteta dominirala izborom. Preopterećenje je povećalo prosečno čekanje na 11,25 taktova, dok je CPU ostao zauzet 89,10%.

Scenario kvara potvrđuje ograničeno ponovno pokretanje i ponovno raspoređivanje. Jedan kvar čvora izazvao je dva ponovna pokretanja i dva nova smeštanja, a svih pet procesa završilo je za 16 taktova. To nije live migracija. Svaki pogođeni proces počeo je novi pokušaj od punog trajanja, što odgovara kodu i prijavljenim metrikama.

Granice simulacije isključuju stvarne udaljene procese, preempciju, trajanje I/O zahteva, mrežnu latenciju, trajnu bazu, autentifikaciju i distribuirani konsenzus. U tim granicama odluka raspoređivača, promena stanja i posledica kvara ostaju neposredno vidljive u kodu, testovima i ponovljivim scenarijima.

5. Spisak literature

Ajmani, S. 2014. *Go Concurrency Patterns: Context*. The Go Programming Language. <https://go.dev/blog/context>.

Ongaro, D. i Ousterhout, J. 2014. *In Search of an Understandable Consensus Algorithm*. USENIX Association. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>.

Silberschatz, A., Galvin, P. B. i Gagne, G. 2014. *Operating System Concepts Essentials, 2nd edition*. Wiley.

Tanenbaum, A. S. i Bos, H. 2015. *Modern Operating Systems, 4th edition*. Pearson.

The Kubernetes Authors. *Assigning Pods to Nodes*. Kubernetes Documentation. <https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/>.

The Kubernetes Authors. *Scheduling Framework*. Kubernetes Documentation. <https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/>.